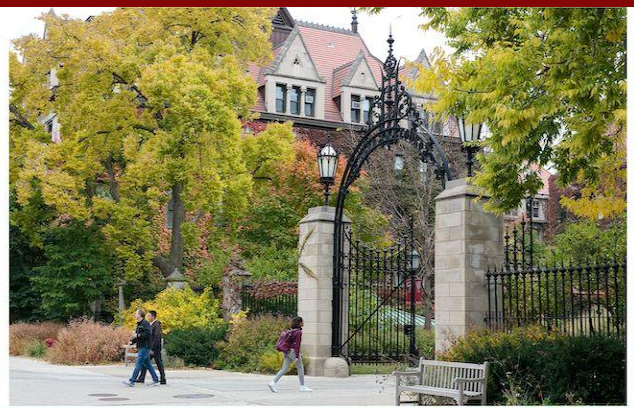
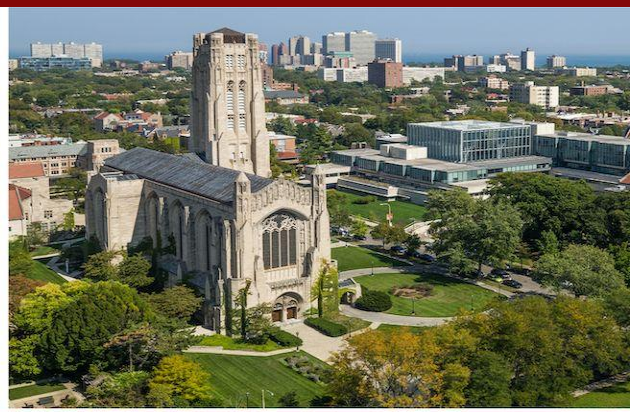
# MPCS 51300 - Compilers
# M5: Register Allocation for Code Generation

Lamont Samuels

Copyright 2010, Keith D. Cooper & Linda Torczon, all rights reserved.
Students enrolled in Comp 412 at Rice University have explicit permission to make copies of these materials for their personal use.
Faculty from other educational institutions may use these materials for nonprofit educational purposes, provided this copyright notice is preserved.

THE UNIVERSITY OF CHICAGO | MASTERS PROGRAM IN COMPUTER SCIENCE

# Single Static Assignment (SSA) Revisited

- SSA is an intermediate representation where each variable is assigned exactly once. This simplifies analysis because a variable's definition is clearly distinct from its uses.
- In SSA, each variable is given a unique version whenever it is redefined.
  - For example, instead of just x, we may have x1, x2, x3, etc., for each definition of x.
- SSA simplifies liveness analysis and other optimizations because the flow of data is more explicit (each variable is defined once and only once).

# Stacked-based Representation in LLVM IR

- In LLVM IR, stack-based representation often uses memory locations and the stack to store variable values.
- Variables are loaded into registers for computation and written back to the stack after operations, which can introduce inefficiency in instruction scheduling and register allocation.
- Stack-based representations often lead to higher memory usage and more load/store operations, which can be inefficient during optimization stages.

```
define i64 @main()
{
L0:
   %_ret_val = alloca i64
   %a = alloca i64
   %b = alloca i64
   %c = alloca i64
   store i64 5, i64* %a
   store i64 7, i64* %b
   %r0 = load i64, i64* %a
   %r1 = load i64, i64* %b
   %r2 = add i64 %r0, %r1
   store i64 %r2, i64* %c
   %r3 = load i64, i64* %c
   call i8 (i8*, ...) @printf(i8* getelementptr inbounds (
                        [7 x i8], [7 x i8]* @.fstr0, i32 0, i32 0),i64 %r3)
   store i64 0, i64* %_ret_val
   br label %L1
L1:
   %r4 = load i64, i64* %_ret_val
   ret i64 %r4

}
```

```
func main() {
  var a, b, c int;
  a = 5;
  b = 7;
  c = a + b;
  printf("c=%d\n",c);

}
```

# Why Convert Stack-Based LLVM IR to Virtual Register SSA?

- What We Have (Stack-Based IR)
  - Variables stored in memory (alloca, load, store)
  - Values repeatedly written to and read from stack slots
  - Not in true SSA form
  - Hard to reason about data dependencies
- What We Want (Virtual Register SSA)
  - Each variable becomes a virtual register
  - Each value defined exactly once (SSA)
  - No unnecessary memory traffic

# φ-Nodes

- When converting from stack-based IR to SSA:
  - Each variable must be assigned exactly once
  - Different control-flow paths may assign different values
  - At merge points, we must reconcile those values
- Without φ:
  - We cannot represent “x depends on which path we took”
  - φ-nodes solve this problem.

# The Problem at a Merge

```
entry:
  store 0 → x
  br cond, then, else

then:
  store 1 → x
  br merge

else:
  store 2 → x
  br merge

merge:
  %y = load x
```

- At merge, what is x?
  - From then: 1
  - From else: 2
- Both are valid depending on control flow.

# SSA Requires Single Assignment

- In SSA:
- Every value is defined exactly once.
- We cannot reassign x.
- So instead of:

```
x = 1
x = 2
```

- We must create distinct names:

```
x1 = 1
x2 = 2
```

- Now at merge, which one is x?
- We need a selector.

# Aside: Phi Instructions

- A LLVM phi instruction is a special instruction used in SSA form to merge different versions of a variable at control flow joins (e.g., at the beginning of basic blocks where multiple paths converge).
  - LLVM phi instructions appear at the entry of a basic block, where there are multiple predecessors, and it merges the incoming values from those predecessors.
  - Specifically, they merge different definitions of a variable that come from multiple control flow paths (e.g., branches or loops).
- Phi instructions are necessary in SSA form to maintain the rule that every variable is assigned exactly once.

```
func foo(x int) int {
    var sum int;
    if (x > 3)
    {
        sum = x + 2;
    } else {
        sum = x + 1;
    }
    return sum;
}
```

Sum will either be x+2 or x+1 based on the control-flow of the value of x.

# Phi Instruction

%result = phi <type> [<value1>, <block1>], [<value2>, <block2>], ...

- <result>: This is the result variable that will hold the merged value.
- <type>: The type of the value being merged (e.g., i64 for 64-bit integers).
- <value1> and <value2>: These are the values coming from different predecessor blocks. Each value corresponds to a definition from a predecessor block.
- <block1> and <block2>: These are the predecessor blocks where each value originates. The control flow determines which value the phi node will pick based on the path taken.

# Phi Instructions

- The number of operands for a phi node depends on how many predecessor blocks a basic block has. Each predecessor contributes an operand (a potential value) for the phi instruction.
- If a basic block has N predecessors, the phi instruction will have N operands (each representing a possible value coming from a predecessor).

# Phi Instruction Example

```
L2:
    %r2 = icmp sgt i64 %x, 3
    br i1 %r2, label %L3, label %L4
L4:
    %r6 = add i64 %x, 1
    br label %L5
L3:
    %r4 = add i64 %x, 2
    br label %L5
L5:
    %sum_1 = phi i64 [ %r4, %L3 ],  [ %r6, %L4 ]
```

```
func foo(x int) int {
    var sum int;
    if (x > 3)
    {
        sum = x + 2;
    } else {
        sum = x + 1;
    }
    return sum;
}
```

# Dataflow Analysis

# Dataflow Analysis

- Dataflow analysis is a technique for computing information that flows through a program's control-flow graph (CFG).
- It answers questions like:
  - What definitions reach this point?
  - What variables are live?
  - What is the current value of a variable?
- It works by:
  - Associating information with each block
  - Propagating that information along CFG edges
  - Iterating until the information stabilizes

# Forward vs Backward Dataflow

- Forward analysis
  - Information flows in direction of control flow
  - Example: reaching definitions, constant propagation
  - Our SSA construction is forward
- Backward analysis
  - Information flows opposite control flow
  - Example: liveness analysis(we'll talk about this when we discuss register allocation and optimizations)

# Defs and Uses

- Def (Definition):
  - A "definition" of a variable occurs when a value is assigned to that variable.

```
store i64 %v, i64* %x    ; defines x
%t = add i64 %a, %b      ; defines %t
```

- Use (Definition):
  - A “use” occurs when a variable is read, but not assigned a new value.

```
%t = load i64, i64* %x   ; uses x
%u = add i64 %a, %b      ; uses %a and %b
```

- A variable must be defined before it can be used.
- In SSA construction:
  - We track definitions of stack variables
  - We replace uses (loads) with their current SSA value

# Aside: What Is a Fixed-Point Algorithm?

- A fixed-point algorithm:
  - Repeatedly updates information
  - Stops when no changes occur
- Formally:
  - A fixed point is reached when applying the algorithm again produces no further changes.
- Why needed?
  - SSA Construction: Loops create cyclic dependencies
  - Information must circulate through cycles
  - One pass is not enough

# SSA via Dataflow (High-Level Idea)

- Let B be a Basic block, lets define IN and OUT maps:
  - IN[B] = variable → SSA value at block entry
  - OUT[B] = variable → SSA value at block exit
- Algorithm idea:
  - Merge predecessor OUT maps to form IN
  - Insert φ when predecessors disagree
  - Process instructions to compute OUT
  - Repeat until stable

# SSA Dataflow Algorithm (Pseudocode)

```
func Mem2Reg(..) {
for each block B:
   OUT[B] = empty map

repeat
   changed = false

   for each block B:

      old_out = copy(OUT[B])

      ------------------------------------------------
      // Step 1: Compute IN[B]
      ------------------------------------------------

      if B is entry:
         IN[B] = default values for stack variables
      else:
         IN[B] = merge(OUT[p] for each predecessor p)

      ------------------------------------------------
      // Step 2: Process block
      ------------------------------------------------

      current = copy(IN[B])
      valueOf = empty map

      for each instruction I in B:

         case I of:

            //Next slide
```

IN[B] : Map<StackVar, SSAValue>
OUT[B] : Map<StackVar, SSAValue>

current : Map<StackVar, SSAValue> // block-local copy of IN
valueOf : Map<Register, SSAValue> // block-local substitution map

```
IN[B] = merge(OUT[p] for p in preds(B))
```

```
vals = { OUT[p][v] for each predecessor p }

if all values identical:
   IN[B][v] = that value
else:
   create ϕ node in B
   IN[B][v] = ϕ_result
```

Key idea:
φ nodes are created when multiple paths disagree on a variable's value.

# SSA Dataflow Algorithm (Pseudocode)

```
        ----------------------------------------
        1) Load from promoted stack variable (v):  %r = load v
        ----------------------------------------

           // Replace memory read with register value
           valueOf[%r] = current[v]

        ----------------------------------------
        2) Store to promoted stack variable:    store %r -> v
        ----------------------------------------
           storedValue = resolve(%r)
           current[v] = storedValue

        ----------------------------------------
        3) Other instruction defining %r :  %r = op operands...
        ----------------------------------------

           valueOf[%r] = %r

    ------------------------------------------------
    // Step 3: Update OUT
    ------------------------------------------------

    OUT[B] = current

    if OUT[B] != old_out:
       changed = true

until changed == false
}
```

```
resolve(x):
   if x is a register and x in valueOf:
      return valueOf[x]
   else:
      return x
```

# What Does "Stable" Mean?

- The algorithm stops when:

```
For all blocks B:
   OUT[B] did not change this iteration
```

- That means:
  - No new φ inserted
  - No mappings updated
  - Information fully propagated
  - We have reached a fixed point.

# Cleanup Pass

- After Convergence:
  - For each instruction:
    - If alloca of promoted variable → remove
    - If load from promoted variable → remove
    - If store to promoted variable → remove
- Now IR is pure virtual-register SSA.

# Transitioning to Register-based Implementation (LLVM IR SSA)

## Stack-based

```
define i64 @main()
{
L0:
   %_ret_val = alloca i64
   %a = alloca i64
   %b = alloca i64
   %c = alloca i64
   store i64 5, i64* %a
   store i64 7, i64* %b
   %r0 = load i64, i64* %a
   %r1 = load i64, i64* %b
   %r2 = add i64 %r0, %r1
   store i64 %r2, i64* %c
   %r3 = load i64, i64* %c
   call i8 (i8*, ...) @printf(i8* getelementptr inbounds (
                              [7 x i8], [7 x i8]* @.fstr0, i32 0, i32 0),i64 %r3)
   store i64 0, i64* %_ret_val
   br label %L1
L1:
   %r4 = load i64, i64* %_ret_val
   ret i64 %r4

}
```

## Register-based

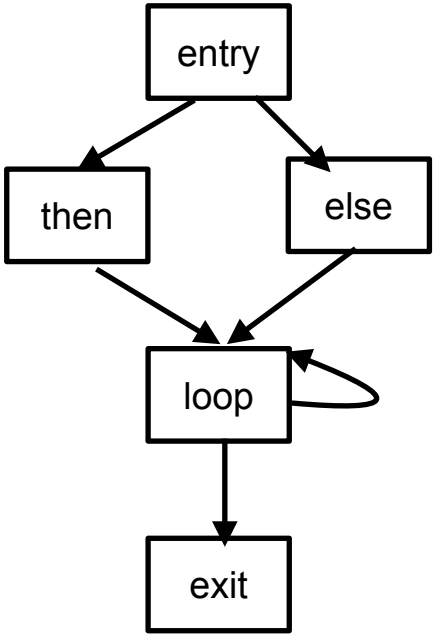
```
define i64 @main()
{
L0:
   %r2 = add i64 5, 7
   call i8 (i8*, ...) @printf(i8* getelementptr inbounds
      ([7 x i8], [7 x i8]* @.fstr0, i32 0, i32 0),i64 %r2)
   br label %L1
L1:
   ret i64 0

}
```

Source code

```
func main() {
 var a, b, c int;
 a = 5;
 b = 7;
 c = a + b;
 printf("c=%d\n",c);

}
```

```
define i64 @example(i64 %cond, i64 %n) {
entry:
  %x = alloca i64
  %y = alloca i64
  %P_cond = alloca i64
  %P_n = alloca i64
  store i64 %cond, i64* %P_cond
  store i64 %n, i64* %P_n
  store i64 0, i64* %x
  store i64 10, i64* %y
  %r0 = load i64, i64* %P_cond
  %r1 = bitcast i64 %r0 to i1
  br i1 %r1, label %then, label %else

then:
  store i64 1, i64* %x
  br label %loop

else:
  store i64 2, i64* %x
  br label %loop

loop:
  %r2 = load i64, i64* %x
  %r3 = load i64, i64* %y
  %r4 = add i64 %r2, 1
  %r5 = add i64 %r3, %r4
  store i64 %r4, i64* %x
  store i64 %r5, i64* %y
  %r6 = load i64, i64* %P_n
  %c = icmp slt i64 %r4, %r6
  br i1 %c, label %loop, label %exit

exit:
  %r7 = load i64, i64* %x
  %r8 = load i64, i64* %y
  %r9 = add i64 %r7, %r8
  ret i64 %r9
}
```

Please note that your compiler might generate a CFG for this program might look different with more merge nodes due to having more exit nodes. The algorithm does not change in that case.




All stack variables will be tracked:
%x, %y, %P_cond, %P_n

```
define i64 @example(i64 %cond, i64 %n) {
entry:
  %x = alloca i64
  %y = alloca i64
  %P_cond = alloca i64
  %P_n = alloca i64
  store i64 %cond, i64* %P_cond
  store i64 %n, i64* %P_n
  store i64 0, i64* %x
  store i64 10, i64* %y
  %r0 = load i64, i64* %P_cond
  %r1 = bitcast i64 %r0 to i1
  br i1 %r1, label %then, label %else

then:
  store i64 1, i64* %x
  br label %loop

else:
  store i64 2, i64* %x
  br label %loop

loop:
  %r2 = load i64, i64* %x
  %r3 = load i64, i64* %y
  %r4 = add i64 %r2, 1
  %r5 = add i64 %r3, %r4
  store i64 %r4, i64* %x
  store i64 %r5, i64* %y
  %r6 = load i64, i64* %P_n
  %c = icmp slt i64 %r4, %r6
  br i1 %c, label %loop, label %exit

exit:
  %r7 = load i64, i64* %x
  %r8 = load i64, i64* %y
  %r9 = add i64 %r7, %r8
  ret i64 %r9
}
```

# Initialization

For all B:
    OUT[B] = {}

OUT[entry] = {}
OUT[then] = {}
OUT[else] = {}
OUT[loop] = {}
OUT[exit] = {}

# Iteration 1

```
entry:
  %x = alloca i64
  %y = alloca i64
  %P_cond = alloca i64
  %P_n = alloca i64
  store i64 %cond, i64* %P_cond
  store i64 %n, i64* %P_n
  store i64 0, i64* %x
  store i64 10, i64* %y
  %r0 = load i64, i64* %P_cond
  %r1 = bitcast i64 %r0 to i1
  br i1 %r1, label %then, label %else
```

OldOut [entry]: {}

IN[entry]:
%x -> 0
%y -> 0
%P_cond -> 0
%P_n -> 0

valueOf:
%r0 -> 0

The final state of current is What out is since Out is a Copy of it.

OUT[entry]:
%x -> 0
%y ->10
%P_cond -> %cond
%P_n -> %n

Out has changed! changed==True

# Iteration 1

```
then:
  store i64 1, i64* %x
  br label %loop
```

OldOut [then]: {}

IN[then]:
%x -> 0
%y ->10
%P_cond -> %cond
%P_n -> %n

valueOf:{}

The final state of current is What out is since Out is a Copy of it.

OUT[then]:
%x -> 1
%y ->10
%P_cond -> %cond
%P_n -> %n

Out has changed! changed==True

# Iteration 1

```
else:
  store i64 2, i64* %x
  br label %loop
```

OldOut [else]: {}

valueOf:{}

The final state of current is What out is since Out is a Copy of it.

IN[else]:
%x -> 0
%y ->10
%P_cond -> %cond
%P_n -> %n

OUT[else]:
%x -> 2
%y ->10
%P_cond -> %cond
%P_n -> %n

Out has changed! changed==True

# Iteration 1

Before

```
loop:
  %r2 = load i64, i64* %x
  %r3 = load i64, i64* %y
  %r4 = add i64 %r2, 1
  %r5 = add i64 %r3, %r4
  store i64 %r4, i64* %x
  store i64 %r5, i64* %y
  %r6 = load i64, i64* %P_n
  %c = icmp slt i64 %r4, %r6
  br i1 %c, label %loop, label %exit
```

After

```
loop:
  %r10 = phi i64 [1, then], [2, else]
  %r2 = load i64, i64* %x
  %r3 = load i64, i64* %y
  %r4 = add i64 %r2, 1
  %r5 = add i64 %r3, %r4
  store i64 %r4, i64* %x
  store i64 %r5, i64* %y
  %r6 = load i64, i64* %P_n
  %c = icmp slt i64 %r4, % %r6
  br i1 %c, label %loop, label %exit
```

OUT[loop]: {}

OldOut [loop]: {}

valueOf:
%r2 -> %r10
%r3 -> 10

OUT[then]:
%x -> 1
%y ->10
%P_cond -> %cond
%P_n -> %n

OUT[else]:
%x -> 2
%y ->10
%P_cond -> %cond
%P_n -> %n

Predecessors:
1. then,
2. else,
3. loop (backedge, but not yet computed)

IN[loop] (start):
%x -> %r10
%y ->10
%P_cond -> %cond
%P_n -> %n

OUT[loop]:
%x -> %r4
%y ->%r5
%P_cond -> %cond
%P_n -> %n

Out has changed! changed==True

# Iteration 1

```
exit:
  %r7 = load i64, i64* %x
  %r8 = load i64, i64* %y
  %r9 = add i64 %r7, %r8
  ret i64 %r9
```

OldOut [exit]: {}

valueOf:
%r7 -> %r4
%r8 -> %r5

IN[exit]:
%x ->%r4
%y ->%r5
%P_cond -> %cond
%P_n -> %n

OUT[exit]:
%x -> %r4
%y ->%r5
%P_cond -> %cond
%P_n -> %n

Out has changed! changed==True

# Iteration 2

```
entry:
  %x = alloca i64
  %y = alloca i64
  %P_cond = alloca i64
  %P_n = alloca i64
  store i64 %cond, i64* %P_cond
  store i64 %n, i64* %P_n
  store i64 0, i64* %x
  store i64 10, i64* %y
  %r0 = load i64, i64* %P_cond
  %r1 = bitcast i64 %r0 to i1
  br i1 %r1, label %then, label %else
```

valueOf:
%r0 -> 0

oldOut[entry]:
%x -> 0
%y ->10
%P_cond -> %cond
%P_n -> %n

IN[entry]:
%x -> 0
%y -> 0
%P_cond -> 0
%P_n -> 0

OUT[entry]:
%x -> 0
%y ->10
%P_cond -> %cond
%P_n -> %n

Out not changed! changed==False

# Iteration 2

valueOf:{}

```
then:
  store i64 1, i64* %x
  br label %loop
```

oldOut[then]:
%x -> 1
%y ->10
%P_cond -> %cond
%P_n -> %n

IN[then]:
%x -> 0
%y ->10
%P_cond -> %cond
%P_n -> %n

OUT[then]:
%x -> 1
%y ->10
%P_cond -> %cond
%P_n -> %n

Out has not changed! changed==False

# Iteration 2

```
else:
  store i64 2, i64* %x
  br label %loop
```

oldOut[else]:
%x -> 2
%y ->10
%P_cond -> %cond
%P_n -> %n

IN[else]:
%x -> 0
%y ->10
%P_cond -> %cond
%P_n -> %n

valueOf: {}

OUT[else]:
%x -> 2
%y ->10
%P_cond -> %cond
%P_n -> %n

Out has not changed! changed==False

# Iteration 2

Before

```
loop:
  %r10 = phi i64 [1, then], [2, else]
  %r2 = load i64, i64* %x
  %r3 = load i64, i64* %y
  %r4 = add i64 %r2, 1
  %r5 = add i64 %r3, %r4
  store i64 %r4, i64* %x
  store i64 %r5, i64* %y
  %r6 = load i64, i64* %P_n
  %c = icmp slt i64 %r4, %P_n
  br i1 %c, label %loop, label %exit
```

After

```
loop:
  %r10 = phi i64 [1, then], [2, else], [%r4, loop]
  %r11 = phi i64 [10, then], [10, else], [%r5, loop]
  %r2 = load i64, i64* %x
  %r3 = load i64, i64* %y
  %r4 = add i64 %r2, 1
  %r5 = add i64 %r3, %r4
  store i64 %r4, i64* %x
  store i64 %r5, i64* %y
  %r6 = load i64, i64* %P_n
  %c = icmp slt i64 %r4, %r6
  br i1 %c, label %loop, label %exit
```

oldOut[loop]:
%x -> %r4
%y ->%r5
%P_cond -> %cond
%P_n -> %n

OUT[then]:
%x -> 1
%y ->10
%P_cond -> %cond
%P_n -> %n

OUT[else]:
%x -> 2
%y ->10
%P_cond -> %cond
%P_n -> %n

Predecessors:
1. then,
2. else,
3. loop (backedge, but not yet computed)

valueOf:
%r2 -> %r10
%r3 -> %r11

IN[loop] (start):
%x -> %r10
%y ->%r11
%P_cond -> %cond
%P_n -> %n

OUT[loop]:
%x -> %r4
%y ->%r5
%P_cond -> %cond
%P_n -> %n

Added Phi! changed==True

# Iteration 2

valueOf:
%r7 -> %r4
%r8 -> %r5

exit:
%r7 = load i64, i64* %x
%r8 = load i64, i64* %y
%r9 = add i64 %r7, %r8
ret i64 %r9

oldOut[exit]:
%x -> %r4
%y ->%r5
%P_cond -> %cond
%P_n -> %n

IN[exit]:
%x ->%r4
%y ->%r5
%P_cond -> %cond
%P_n -> %n

OUT[exit]:
%x -> %r4
%y ->%r5
%P_cond -> %cond
%P_n -> %n

Out has not changed But changed is still True

- Not going to show you this but if you were to run the algorithm again you would reach convergence.

# After convergence

- For each block B:

```
for each block B:

   current = copy(IN[B])
   valueOf = empty map

   for each instruction I in B:

      case I of:

         load v:
            replace all uses of %r with current[v]
            delete instruction

         store %r -> v:
            current[v] = resolve(%r)
            delete instruction

         %r = op operands:
            rewrite operands using resolve
            valueOf[%r] = %r

         otherwise:
            rewrite operands if needed
```

current = copy(IN[B])

for each instruction in original block order:

if the instruction is a stack load:

If the load corresponds to a stack-based load, update the current map to assign the value of the stack variable to the destination register. Do not remove load instructions that are storing to an address not on the stack!

If load is not a stack-based load make sure to still check the map in case the operand is mapped to a different register.

Remove the load instruction after updating the map.

if the instruction is a stack store:

If the store corresponds to a stack-based store (i.e., writing to memory), update the map with the new value stored in the variable. Do not remove store instructions that are storing to an address not on the stack!

If store is not a stack-based load make sure to still check the map in case the operand is mapped to a different register.

otherwise:

Examine the defs (definitions) and uses of the instruction.

If an operand corresponds to a variable already in the map, update the operand to reflect the value mapped to that variable. Otherwise, if it's not in the map do not modify that operand in the instruction.

# After Convergence

entry:

~~%x = alloca i64~~
~~%y = alloca i64~~
~~%P_cond = alloca i64~~
~~%P_n = alloca i64~~
~~store i64 %cond, i64* %P_cond~~
~~store i64 %n, i64* %P_n~~
~~store i64 0, i64* %x~~
~~store i64 10, i64* %y~~
~~%r0 = load i64, i64* %P_cond~~
%r1 = bitcast i64 %n to i1
br i1 %r1, label %then, label %else

Current[entry]:
%x -> 0
%y -> 0
%P_cond -> 0
%P_n -> 0

valueOf:
%r0 -> %n

# After Convergence

then:

~~store i64 1, i64* %x~~

br label %loop

Current[then]:
%x -> 0
%y ->10
%P_cond -> %cond
%P_n -> %n

valueOf: {}

# After Convergence

else:

~~store i64 2, i64* %x~~

br label %loop

Current[else]:
%x -> 0
%y ->10
%P_cond -> %cond
%P_n -> %n

valueOf: {}


THE UNIVERSITY OF CHICAGO | MASTERS PROGRAM IN COMPUTER SCIENCE

# After Convergence

Before

```
loop:
  %r10 = phi i64 [1, then], [2, else], [%r4, loop]
  %r11 = phi i64 [10, then], [10, else], [%r5, loop]
  %r2 = load i64, i64* %x
  %r3 = load i64, i64* %y
  %r4 = add i64 %r2, 1
  %r5 = add i64 %r3, %r4
  store i64 %r4, i64* %x
  store i64 %r5, i64* %y
  %r6 = load i64, i64* %P_n
  %c = icmp slt i64 %r4, %r6
  br i1 %c, label %loop, label %exit
```

After

loop:
  %r10 = phi i64 [1, then], [2, else], [%r4, loop]
  %r11 = phi i64 [10, then], [10, else], [%r5, loop]
  ~~%r2 = load i64, i64* %x~~
  ~~%r3 = load i64, i64* %y~~
  %r4 = add i64 %r10, 1
  %r5 = add i64 %r11, %r4
  ~~store i64 %r4, i64* %x~~
  ~~store i64 %r5, i64* %y~~
  ~~%r6 = load i64, i64* %P_n~~
  %c = icmp slt i64 %r4, %cond
  br i1 %c, label %loop, label %exit

Current[loop]:
- %x -> %r10
- %y ->%r11
- %P_cond -> %cond
- %P_n -> %n

valueOf:
- %r2 ->%r10
- %r3 -> %r11
- %r6 -> %cond

# After Convergence

exit:

~~%r7 = load i64, i64* %x~~

~~%r8 = load i64, i64* %y~~

%r9 = add i64 %r4, %r5

ret i64 %r9

Current[exit]:
%x ->%r4
%y ->%r5
%P_cond -> %cond
%P_n -> %n

valueOf:
%r7 ->%r4
%r8 ->%r5

- Done with rewrite.

```
define i64 @example(i64 %cond, i64 %n) {
entry:
  %x = alloca i64
  %y = alloca i64
  %P_cond = alloca i64
  %P_n = alloca i64
  store i64 %cond, i64* %P_cond
  store i64 %n, i64* %P_n
  store i64 0, i64* %x
  store i64 10, i64* %y
  %r0 = load i64, i64* %P_cond
  %r1 = bitcast i64 %r0 to i1
  br i1 %r1, label %then, label %else

then:
  store i64 1, i64* %x
  br label %loop

else:
  store i64 2, i64* %x
  br label %loop

loop:
  %r2 = load i64, i64* %x
  %r3 = load i64, i64* %y
  %r4 = add i64 %r2, 1
  %r5 = add i64 %r3, %r4
  store i64 %r4, i64* %x
  store i64 %r5, i64* %y
  %r6 = load i64, i64* %P_n
  %c = icmp slt i64 %r4, %r6
  br i1 %c, label %loop, label %exit

exit:
  %r7 = load i64, i64* %x
  %r8 = load i64, i64* %y
  %r9 = add i64 %r7, %r8
  ret i64 %r9
}
```

```
define i64 @example(i64 %cond, i64 %n) {
entry:
  %r1 = bitcast i64 %n to i1
  br i1 %r1, label %then, label %else

then:
  br label %loop

else:
  br label %loop

loop:
  %r10 = phi i64 [1, then], [2, else], [%r4, loop]
  %r11 = phi i64 [10, then], [10, else], [%r5, loop]
  %r4 = add i64 %r10, 1
  %r5 = add i64 %r11, %r4
  %c = icmp slt i64 %r4, %cond
  br i1 %c, label %loop, label %exit

exit:
  %r9 = add i64 %r4, %r5
  ret i64 %r9
}
```

# Aside: Read Statement

- In Golite, the read statement (scan x) reads a single integer value into a variable (i.e., the address of the variable). This variable may be a stack variable. Since we are removing stack allocation, we need a temporary location to store stack variable reads.
  - The solution is to use a global scratch space (i.e., a global memory address) to store the value of the stack variable.
  - You then load the value from this global variable into a new temporary register.
  - Update the map so that the new temporary register represents the current value of the stack variable.

# Aside: Read Statement

- First define a new global variable for reading stack variables. Normal convention is to call this scratch space.

```
@.read_scratch = common global i64 0
```

- Below is the translation:

```
func  main() {
    var x int;
    scan x;
    printf("%d",x);
}
```

Stack-based

```
define i64 @main()
{
L0:
    %_ret_val = alloca i64
    store i64 0, i64* %_ret_val
    %x = alloca i64
    call i8 (i8*, ...) @scanf(... * @.read, i32 0, i32 0),i64* %x)
    %r0 = load i64, i64* %x
    call i8 (i8*, ...) @printf(.. * @.fstr0, i32 0, i32 0),i64 %r0)
    ...
```

Note showing "…" represents since its too long for the slide.

Register-based

```
@.read_scratch = common global i64 0

define i64 @main()
{
L0:
    call i8 (i8*, ...) @scanf(... * @.read, i32 0, i32 0),i64* @.read_scratch)
    %r2 = load i64, i64* @.read_scratch
    call i8 (i8*, ...) @printf(... * @.fstr0, i32 0, i32 0),i64 %r2)
    br label %L1
L1:
    ret i64 0

}
@.fstr0 = private unnamed_addr constant [4 x i8] c"%ld\00", align 1
@.read = private unnamed_addr constant [4 x i8] c"%ld\00", align 1
```

# Liveness Analysis

# What is Liveness Analysis?

- A technique used in compiler optimizations to determine which variables are live at each point in a program. A variable is considered live
  - If its value may be needed in the future (i.e., if it will be used before it is overwritten).
  - Generally speaking, it is live between its definition and its last use.
- Liveness analysis helps identify opportunities for optimization, such as register allocation or dead code elimination.
  - In this lecture, we will focus on register allocation and come back to liveness analysis when talking about other optimizations.

# Live Ranges

- The live range represent the span of time a variable is live in a program.
- The live range begins at the point where the variable is first used and ends when it is last used or overwritten.
  - If a variable is defined an instruction $i_x$ and last used at an instruction $i_y$ then it's live range is $[i_x, i_j]$

# Live Ranges

```
func foo(a int, b int) int
var c int;
c= b+a*2;
return c;
```

```
1. define i64 @foo (i64 %a, i64 %b)
2. {
3. L1:
4.  %foo_retval = alloca i64
5. %P a = alloca i64
6. %P_b = alloca i64
7. %c = alloca i64
8. store i64 %a, i64*%P_a
9. store i64 %b, i64*%P_b
10. %rO = load i64, i64* %P_a
11. %r1 = mul i64 %ro, 2
12. %r2 = load i64, i64* %P_b
13. %r3 = add i64 %r2, %r1
14. store i64 %r3, i64* %c
15. br label %L2
16. L2:
17. %r4 = load i64, i64* %c
18.store i64 %r4, i64* %foo_retval
29. %r5 = load i64, i64* %foo_retval
20. ret i64 %r5
21. }
```

# Live Ranges

```
1. define i64 @foo (i64 %a, i64 %b)
2. {
3. L1:
4.  %foo_retval = alloca i64
5. %P a = alloca i64
6. %P_b = alloca i64
7. %c = alloca i64
8. store i64 %a, i64*%P_a
9. store i64 %b, i64*%P_b
10. %rO = load i64, i64* %P_a
11. %r1 = mul i64 %ro, 2
12. %r2 = load i64, i64* %P_b
13. %r3 = add i64 %r2, %r1
14. store i64 %r3, i64* %c
15. br label %L2
16. L2:
17. %r4 = load i64, i64* %c
18.store i64 %r4, i64* %foo_retval
29. %r5 = load i64, i64* %foo_retval
20. ret i64 %r5
21. }
```

| Virtual Register | Live Start | Live End |
|---|---|---|
| a | 1 | 8 |
| b | 1 | 9 |
| foo_retval | 4 | 19 |
| P_a | 5 | 10 |
| P_b | 6 | 12 |
| c | 7 | 17 |
| r0 | 10 | 11 |
| r1 | 11 | 13 |
| r2 | 12 | 13 |
| r3 | 13 | 14 |
| r4 | 17 | 18 |
| r5 | 19 | 20 |

# Register Allocation: Linear Scan Allocation

# Register Allocation

- LLVM uses an infinite number of registers when generating code but when translating too assembly we only have a finite number of registers to use; therefore, we need to have use a register allocation scheme
  - A process that assigned variables and temporaries to registers and manages the transfer of data in and out of registers.
- Two main approaches:
  - Stack Based Model
  - Global Register Allocation

# Global Register Allocation

- Two popular techniques for global register allocation, which works on one (entire) procedure at a time:
- Linear Scan Allocator: Allocates registers to variables in a single linear-time scan
  - Considerably faster than algorithms based on graph coloring
  - Focuses on making compile time faster but could produce a poor register allocation.
  - Paper: “Linear Scan Register Allocation” by M. Poletto and V. Sarkar.
  - LLVM uses this as one of their allocation schemes
- Register Allocation via Graph Coloring:
  - The state of the art algorithm for allocation
  - Can take a considerable amount of time if heuristics are not chosen correctly.

# Preparing LLVM Register Based IR for Register Allocation

- In LLVM SSA:

```
%x1 = ...
%x2 = ...
%x3 = add i64 %x1, %x2
```

  - Each assignment creates a new version
  - Infinite register names
  - No reuse
- In ARM64:
  - Only 31 general-purpose registers:
    - x0–x30
  - Some reserved:
    - x29 = frame pointer
    - x30 = link register
    - sp = stack pointer
  - We must fit many virtual registers into ~28 usable physical registers.

# Step 1: Removing Φ-Nodes (Destroying SSA also known as Out of SSA)

- A φ is just a copy inserted along incoming edges.

```
B4:
  x3 = φ(x1, x2)
```

- Insert

```
B2:
  x3 = x1
  goto B4

B3:
  x3 = x2
  goto B4
```

- Then remove φ.

# How to Eliminate Φ-Nodes

- SSA Example:

```
if cond:
    x1 = 1
else:
    x2 = 2
```

- CFG Form:

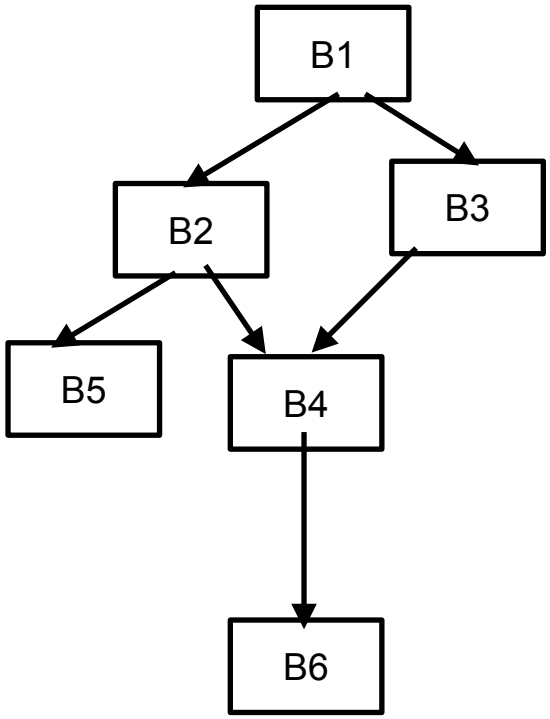
```
  B1
 /  \
B2  B3
 \  /
  B4
```

- B4
  - x3 = φ(x1, x2)

# Critical Edge Problem

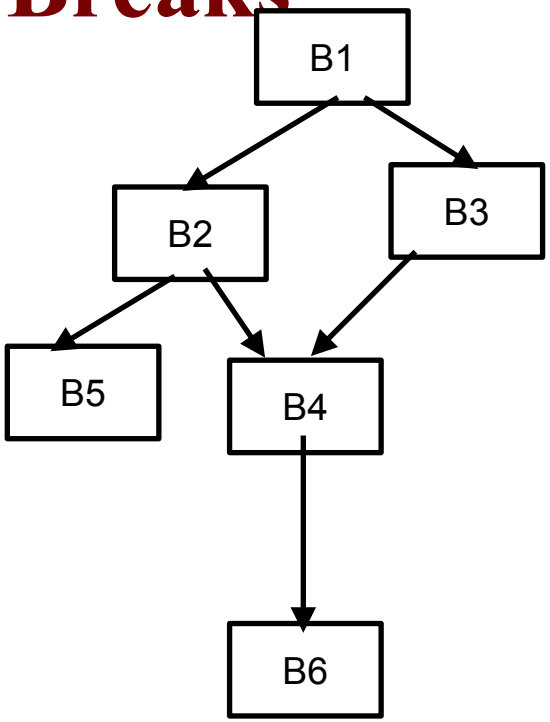
- In a control-flow graph (CFG), An edge A → B is a critical edge if:
  - A has multiple successors
  - B has multiple predecessors
- Formally:
  - |succ(A)| > 1 AND |pred(B)| > 1
- Why Do We Care?
  - Because when removing φ-nodes, we must insert copies on specific edges.
  - If the edge is critical, we have no safe place to put the copy without changing semantics.

# CFG With a Critical Edge




- B1 has 2 successors → (B2, B3)
- B4 has 2 predecessors → (B2, B3)
- Now look at edge: B2 →B4
  - Check: |succ(B2)| > 1 AND |pred(B4)| > 1
  - B2 has multiple successors (B4, B5)
  - B4 has multiple predecessors
  - That edge is critical.

# Why φ-Removal Breaks




- Suppose we have:
  B4:
    x3 = φ(x1 from B2, x2 from B3)
- To remove φ, we want:
  In B2:
    x3 = x1
  In B3:
    x3 = x2
- But here is the problem, If we put:
  B2:
    x3 = x1
    ...
    if (...) goto B4 else goto B5

- **Error: Now we've assigned x3 even if we go to B5. That is WRONG.**
- We only want x3 = x1 executed when going to B4.
- But B2 has two outgoing edges:
  - B2 → B4
  - B2 → B5
- We need to place the copy only on the B2 → B4 path.
- But there is no location that belongs exclusively to that edge. That is the core problem.

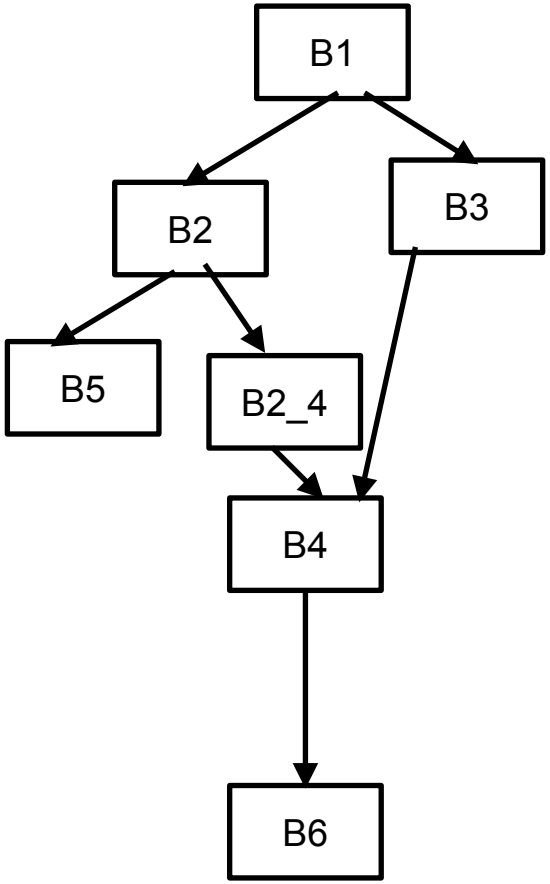
# Critical Edge Solution: Split the Critical Edge

- We create a new block.
- Transform: B2 → B4
- Into: B2 → B2_4 → B4
- Now we have:
  - B2_4 has exactly 1 predecessor
  - B2_4 has exactly 1 successor
  - It belongs purely to that edge.
- Now we can safely insert:

  B2_4:

  x3 = x1

  goto B4





THE UNIVERSITY OF CHICAGO | MASTERS PROGRAM IN COMPUTER SCIENCE

# After Φ Removal

- Now we have:
  - Normal assignments only
  - No φ instructions
  - Still infinite virtual registers
- Now we can do:
  - Register Allocation

# Linear Scan Register Allocation (Step 1: Compute Live Intervals)

Even though SSA guarantees each virtual register is defined only once, you still need to determine where each value is "alive." You do this by:

- A) Make sure your SSA is in Out-of-SSA form:
  - Insert Pseudo-LLVM Move Instruction ( e.g. **mov %op_reg, %dest_reg**). You need to define this instruction.
  - For each PHI operand, insert a pseudo LLVM move at the end of its corresponding predecessor block. This move transfers the value from the PHI's operand's assigned physical register to the physical register designated for the Phi's destination register, effectively converting the IR out of SSA form. Make sure to consider the critical edge special case.
  - ***After updating, the LLVM IR will no longer be valid LLVM IR!***
- B) Linearizing the code:
  - Assign each instruction a unique position (By performing a Reverse Postorder over the CFG for each function and assigning each instruction a unique position).
- C) Compute live ranges for each virtual register
  - *Live Range Start*: The position where a virtual register is defined.
  - *Live Range End*: The position of its last use across all blocks.

# Linear Scan Register Allocation (Step 2: Perform Linear Scan)

Once you have live intervals for each SSA register, you perform the linear scan allocation:

- A) Sort Intervals:
  - Arrange the live intervals in increasing order of their start points.
- B) Keep an active set, which is the set of virtual registers currently assigned to a physical register. As you iterate through the intervals:
  - Expire Old Intervals: Remove intervals from the active set that have ended (i.e., their end point is before the current interval's start).
  - Assign a Register:
    - If a free register is available, assign it to the current interval.
    - If not, decide which interval to spill (often the one with the farthest end point) or split the current interval if beneficial.

# Determining number of Registers

- How many physical registers do you have? It depends on the target architecture.
  - The amount you have available is all registers minus the reserved registers for the architecture.
  - ARM64 has at least 7 reserved registers: {x29, x30, sp, SPILL_REG1, SPILL_REG2, x16, x17, x18}, where SPILL_REG1 and SPILL_RE21 is discussed below.
- Spilling code involves going back to the stack-based representation where you need loads/stores directly to the stack.
  - You need at least two physical register to be dedicated to spilling registers. For ARM64, I recommend you reserve SPILL_REG{1,2} = **x9,x10**.

**Original Source**

```
func foo(a int, b int) int {
    var c,d int;
    scan c;
    scan d;
    if( c > d) {
        d = c + 1;
    } else {
        c = d + 1;
    }
    return a + c + d;
}
```

**LLVM Register-Based**

```
define i64 @foo(i64 %a, i64 %b)
{
L0:
    call i8 (i8*, ...) @scanf(...,i64* @.read_scratch)
    %r15 = load i64, i64* @.read_scratch
    call i8 (i8*, ...) @scanf(..,i64* @.read_scratch)
    %r16 = load i64, i64* @.read_scratch
    br label %L2
L2:
    %r3 = icmp sgt i64 %r15, %r16
    br i1 %r3, label %L3, label %L4
L4:
    %r7 = add i64 %r16, 1
    br label %L5
L3:
    %r5 = add i64 %r15, 1
    br label %L5
L5:
   %c_1 = phi i64 [ %r15, %L3 ], [ %r7, %L4 ]
    %d_1 = phi i64 [ %r5, %L3 ], [ %r16, %L4 ]
    %r10 = add i64 %a, %c_1
    %r12 = add i64 %r10, %d_1
    br label %L1
L1:
    ret i64 %r12

}
```

## Step 1a:Out-of-SSA

```
define i64 @foo(i64 %a, i64 %b)
{
L0:
   call i8 (i8*, ...) @scanf(...,i64* @.read_scratch)
   %r15 = load i64, i64* @.read_scratch
   call i8 (i8*, ...) @scanf(..,i64* @.read_scratch)
   %r16 = load i64, i64* @.read_scratch
   br label %L2
L2:
   %r3 = icmp sgt i64 %r15, %r16
   br i1 %r3, label %L3, label %L4
L4:
   %r7 = add i64 %r16, 1
   br label %L5
L3:
   %r5 = add i64 %r15, 1
   br label %L5
L5:
  %c_1 = phi i64 [ %r15, %L3 ], [ %r7, %L4 ]
   %d_1 = phi i64 [ %r5, %L3 ], [ %r16, %L4 ]
   %r10 = add i64 %_P_a_1, %c_1
   %r12 = add i64 %r10, %d_1
   br label %L1
L1:
   ret i64 %r12

}
```

```
define i64 @foo(i64 %a, i64 %b)
{
L0:
   call i8 (i8*, ...) @scanf(...,i64* @.read_scratch)
   %r15 = load i64, i64* @.read_scratch
   call i8 (i8*, ...) @scanf(..,i64* @.read_scratch)
   %r16 = load i64, i64* @.read_scratch
   br label %L2
L2:
   %r3 = icmp sgt i64 %r15, %r16
   br i1 %r3, label %L3, label %L4
L4:
   %r7 = add i64 %r16, 1
    %c_1 = %r7
    %d_1 = %r16
   br label %L5
L3:
   %r5 = add i64 %r15, 1
    %c_1 = %r15
    %d_1 = %r5
   br label %L5
L5:
  %r10 = add i64 %a, %c_1
   %r12 = add i64 %r10, %d_1
   br label %L1
L1:
   ret i64 %r12

}
```

## *Step 1b:Linearizing the code:*

```
define i64 @foo(i64 %a, i64 %b)
{
L0:
   call i8 (i8*, ...) @scanf(...,i64* @.read_scratch)
   %r15 = load i64, i64* @.read_scratch
   call i8 (i8*, ...) @scanf(..,i64* @.read_scratch)
   %r16 = load i64, i64* @.read_scratch
   br label %L2
L2:
   %r3 = icmp sgt i64 %r15, %r16
   br i1 %r3, label %L3, label %L4
L4:
   %r7 = add i64 %r16, 1
    %c_1 = %r7
    %d_1 = %r16
   br label %L5
L3:
   %r5 = add i64 %r15, 1
    %c_1 = %r15
    %d_1 = %r5
   br label %L5
L5:
  %r10 = add i64 %a, %c_1
   %r12 = add i64 %r10, %d_1
   br label %L1
L1:
   ret i64 %r12

}
```

```
1. define i64 @foo(i64 %a, i64 %b)
{
L0:
 2.  call i8 (i8*, ...) @scanf(...,i64* @.read_scratch)
 3.  %r15 = load i64, i64* @.read_scratch
 4.  call i8 (i8*, ...) @scanf(..,i64* @.read_scratch)
 5. %r16 = load i64, i64* @.read_scratch
 6. br label %L2
L2:
 7.  %r3 = icmp sgt i64 %r15, %r16
8.   br i1 %r3, label %L3, label %L4
L4:
 9. %r7 = add i64 %r16, 1
 10. %c_1 = %r7
 11. %d_1 = %r16
 12.  br label %L5
L3:
 13. %r5 = add i64 %r15, 1
 14. %c_1 = %r15
 15. %d_1 = %r5
 16. br label %L5
L5:
 17. %r10 = add i64 %a, %c_1
 18. %r12 = add i64 %r10, %d_1
 19. br label %L1
L1:
 20. ret i64 %r12

}
```

## *Step 1c: Live Ranges*

```
1. define i64 @foo(i64 %a, i64 %b)
{
L0:
 2.   call i8 (i8*, ...) @scanf(...,i64* @.read_scratch)
 3.   %r15 = load i64, i64* @.read_scratch
 4.   call i8 (i8*, ...) @scanf(..,i64* @.read_scratch)
 5.  %r16 = load i64, i64* @.read_scratch
 6. br label %L2
L2:
 7.   %r3 = icmp sgt i64 %r15, %r16
 8.   br i1 %r3, label %L3, label %L4
L4:
 9.  %r7 = add i64 %r16, 1
 10.  %c_1 = %r7
 11.  %d_1 = %r16
 12.   br label %L5
L3:
 13.  %r5 = add i64 %r15, 1
 14.  %c_1 = %r15
 15. %d_1 = %r5
 16.  br label %L5
L5:
 17. %r10 = add i64 %a, %c_1
 18. %r12 = add i64 %r10, %d_1
 19.  br label %L1
L1:
 20.  ret i64 %r12

}
```

| Virtual Register | Start | End |
|---|---|---|
| a | 1 | 17 |
| b | 1 | -1 |
| r15 | 3 | 14 |
| r16 | 5 | 11 |
| r3 | 7 | 8 |
| r7 | 9 | 10 |
| c_1 | 10 | 17 |
| d_1 | 11 | 18 |
| r5 | 13 | 15 |
| r10 | 17 | 18 |
| r12 | 18 | 20 |

## *Step 2a: Sort Live Ranges*

```
1. define i64 @foo(i64 %a, i64 %b)
{
L0:
 2.   call i8 (i8*, ...) @scanf(...,i64* @.read_scratch)
 3.   %r15 = load i64, i64* @.read_scratch
 4.   call i8 (i8*, ...) @scanf(..,i64* @.read_scratch)
 5.  %r16 = load i64, i64* @.read_scratch
 6. br label %L2
L2:
 7.   %r3 = icmp sgt i64 %r15, %r16
 8.   br i1 %r3, label %L3, label %L4
L4:
 9.  %r7 = add i64 %r16, 1
 10.  %c_1 = %r7
 11.  %d_1 = %r16
 12.   br label %L5
L3:
 13.  %r5 = add i64 %r15, 1
 14.  %c_1 = %r15
 15. %d_1 = %r5
 16.  br label %L5
L5:
 17. %r10 = add i64 %a, %c_1
 18. %r12 = add i64 %r10, %d_1
 19.  br label %L1
L1:
 20.  ret i64 %r12

}
```

| Virtual Register | Start | End |
|---|---|---|
| a | 1 | 17 |
| b | 1 | -1 |
| r15 | 3 | 14 |
| r16 | 5 | 11 |
| r3 | 7 | 8 |
| r7 | 9 | 10 |
| c_1 | 10 | 17 |
| d_1 | 11 | 18 |
| r5 | 13 | 15 |
| r10 | 17 | 18 |
| r12 | 18 | 20 |

## *Step 2b: Perform Linear Scan*

Assume we only have three physical registers: x0, x1, x2

| Virtual Register | Start | End | Allocation location |
|---|---|---|---|
| a | 1 | 17 | |
| b | 1 | -1 | |
| r15 | 3 | 14 | |
| r16 | 5 | 11 | |
| r3 | 7 | 8 | |
| r7 | 9 | 10 | |
| c_1 | 10 | 17 | |
| d_1 | 11 | 18 | |
| r5 | 13 | 15 | |
| r10 | 17 | 18 | |
| r12 | 18 | 20 | |

Current Start Interval = 0

**Active Set** = {}

## *Step 2b: Perform Linear Scan*

Assume we only have three physical registers: x0, x1, x2

| Virtual Register | Start | End | Allocation location |
|---|---|---|---|
| a | 1 | 17 | x0 |
| b | 1 | -1 | |
| r15 | 3 | 14 | |
| r16 | 5 | 11 | |
| r3 | 7 | 8 | |
| r7 | 9 | 10 | |
| c_1 | 10 | 17 | |
| d_1 | 11 | 18 | |
| r5 | 13 | 15 | |
| r10 | 17 | 18 | |
| r12 | 18 | 20 | |

Current Start Interval = 1

**Active Set** = {(a,x0)}

## *Step #2: Perform Linear Scan*

Assume we only have three physical registers: x0, x1, x2

| Virtual Register | Start | End | Allocation location |
|---|---|---|---|
| a | 1 | 17 | x0 |
| b | 1 | -1 | x1 |
| r15 | 3 | 14 | |
| r16 | 5 | 11 | |
| r3 | 7 | 8 | |
| r7 | 9 | 10 | |
| c_1 | 10 | 17 | |
| d_1 | 11 | 18 | |
| r5 | 13 | 15 | |
| r10 | 17 | 18 | |
| r12 | 18 | 20 | |

Current Start Interval = 1

**Active Set** = {(a,x0), (b,x1)}

## *Step #2: Perform Linear Scan*

Assume we only have three physical registers: x0, x1, x2

| Virtual Register | Start | End | Allocation location |
|---|---|---|---|
| a | 1 | 17 | x0 |
| b | 1 | -1 | x1 |
| r15 | 3 | 14 | x1 |
| r16 | 5 | 11 | |
| r3 | 7 | 8 | |
| r7 | 9 | 10 | |
| c_1 | 10 | 17 | |
| d_1 | 11 | 18 | |
| r5 | 13 | 15 | |
| r10 | 17 | 18 | |
| r12 | 18 | 20 | |

Current Start Interval = 3

**Active Set** = {(a,x0), (b,x1)}

Remove b since its end is less than current start interval

**Active Set** = {(a,x0),(r15,x1)}

## *Step #2: Perform Linear Scan*

Assume we only have three physical registers: x0, x1, x2

| Virtual Register | Start | End | Allocation location |
|---|---|---|---|
| a | 1 | 17 | x0 |
| b | 1 | -1 | x1 |
| r15 | 3 | 14 | x1 |
| r16 | 5 | 11 | |
| r3 | 7 | 8 | |
| r7 | 9 | 10 | |
| c_1 | 10 | 17 | |
| d_1 | 11 | 18 | |
| r5 | 13 | 15 | |
| r10 | 17 | 18 | |
| r12 | 18 | 20 | |

Current Start Interval = 3

**Active Set** = {(a,x0), (b,x1)}

Remove b since its end is less than current start interval

**Active Set** = {(a,x0),(r15,x1)}

## *Step #2: Perform Linear Scan*

Assume we only have three physical registers: x0, x1, x2

| Virtual Register | Start | End | Allocation location |
|---|---|---|---|
| a | 1 | 17 | x0 |
| b | 1 | -1 | x1 |
| r15 | 3 | 14 | x1 |
| r16 | 5 | 11 | x2 |
| r3 | 7 | 8 | |
| r7 | 9 | 10 | |
| c_1 | 10 | 17 | |
| d_1 | 11 | 18 | |
| r5 | 13 | 15 | |
| r10 | 17 | 18 | |
| r12 | 18 | 20 | |

Current Start Interval = 5

**Active Set** = {(a,x0),(r15,x1), (r16,x2)}

## *Step #2: Perform Linear Scan*

Assume we only have three physical registers: x0, x1, x2

| Virtual Register | Start | End | Allocation location |
|---|---|---|---|
| a | 1 | 17 | x0 |
| b | 1 | -1 | x1 |
| r15 | 3 | 14 | x1 |
| r16 | 5 | 11 | x2 |
| r3 | 7 | 8 | spill |
| r7 | 9 | 10 | |
| c_1 | 10 | 17 | |
| d_1 | 11 | 18 | |
| r5 | 13 | 15 | |
| r10 | 17 | 18 | |
| r12 | 18 | 20 | |

Current Start Interval = 7

**Active Set** = {(a,x0),(r15,x1), (r16,x2)}

No available registers so we spill.

**Active Set** = {(a,x0),(r15,x1), (r16,x2)}

## *Step #2: Perform Linear Scan*

Assume we only have three physical registers: x0, x1, x2

| Virtual Register | Start | End | Allocation location |
|---|---|---|---|
| a | 1 | 17 | x0 |
| b | 1 | -1 | x1 |
| r15 | 3 | 14 | x1 |
| r16 | 5 | 11 | x2 |
| r3 | 7 | 8 | spill |
| r7 | 9 | 10 | spill |
| c_1 | 10 | 17 | |
| d_1 | 11 | 18 | |
| r5 | 13 | 15 | |
| r10 | 17 | 18 | |
| r12 | 18 | 20 | |

Current Start Interval = 9

**Active Set** = {(a,x0),(r15,x1), (r16,x2)}

No available registers so we spill.

**Active Set** = {(a,x0),(r15,x1), (r16,x2)}

## *Step #2: Perform Linear Scan*

Assume we only have three physical registers: x0, x1, x2

| Virtual Register | Start | End | Allocation location |
|---|---|---|---|
| a | 1 | 17 | x0 |
| b | 1 | -1 | x1 |
| r15 | 3 | 14 | x1 |
| r16 | 5 | 11 | x2 |
| r3 | 7 | 8 | spill |
| r7 | 9 | 10 | spill |
| c_1 | 10 | 17 | spill |
| d_1 | 11 | 18 | |
| r5 | 13 | 15 | |
| r10 | 17 | 18 | |
| r12 | 18 | 20 | |

Current Start Interval = 10

**Active Set** = {(a,x0),(r15,x1), (r16,x2)}

No available registers so we spill.

**Active Set** = {(a,x0),(r15,x1), (r16,x2)}

## *Step #2: Perform Linear Scan*

Assume we only have three physical registers: x0, x1, x2

| Virtual Register | Start | End | Allocation location |
|---|---|---|---|
| a | 1 | 17 | x0 |
| b | 1 | -1 | x1 |
| r15 | 3 | 14 | x1 |
| r16 | 5 | 11 | x2 |
| r3 | 7 | 8 | spill |
| r7 | 9 | 10 | spill |
| c_1 | 10 | 17 | spill |
| d_1 | 11 | 18 | spill |
| r5 | 13 | 15 | |
| r10 | 17 | 18 | |
| r12 | 18 | 20 | |

Current Start Interval = 11

**Active Set** = {(a,x0),(r15,x1), (r16,x2)}

No available registers so we spill.

**Active Set** = {(a,x0),(r15,x1), (r16,x2)}

## *Step #2: Perform Linear Scan*

Assume we only have three physical registers: x0, x1, x2

| Virtual Register | Start | End | Allocation location |
|---|---|---|---|
| a | 1 | 17 | x0 |
| b | 1 | -1 | x1 |
| r15 | 3 | 14 | x1 |
| r16 | 5 | 11 | x2 |
| r3 | 7 | 8 | spill |
| r7 | 9 | 10 | spill |
| c_1 | 10 | 17 | spill |
| d_1 | 11 | 18 | spill |
| r5 | 13 | 15 | x2 |
| r10 | 17 | 18 | |
| r12 | 18 | 20 | |

Current Start Interval = 13

**Active Set** = {(a,x0),(r15,x1), (r16,x2)}

Remove r16 since its end is less than current start interval

**Active Set** = {(a,x0),(r15,x1), (r5,x2)}

## *Step #2: Perform Linear Scan*

Assume we only have three physical registers: x0, x1, x2

| Virtual Register | Start | End | Allocation location |
|---|---|---|---|
| a | 1 | 17 | x0 |
| b | 1 | -1 | x1 |
| r15 | 3 | 14 | x1 |
| r16 | 5 | 11 | x2 |
| r3 | 7 | 8 | spill |
| r7 | 9 | 10 | spill |
| c_1 | 10 | 17 | spill |
| d_1 | 11 | 18 | spill |
| r5 | 13 | 15 | x2 |
| r10 | 17 | 18 | x1 |
| r12 | 18 | 20 | |

Current Start Interval = 17

**Active Set** = {(a,x0),(r15,x1), (r5,x2)}

Remove r5,r15 since its end is less than current start interval

**Active Set** = {(a,x0),(r10,x1)}

## *Step #2: Perform Linear Scan*

Assume we only have three physical registers: x0, x1, x2

| Virtual Register | Start | End | Allocation location |
|---|---|---|---|
| a | 1 | 17 | x0 |
| b | 1 | -1 | x1 |
| r15 | 3 | 14 | x1 |
| r16 | 5 | 11 | x2 |
| r3 | 7 | 8 | spill |
| r7 | 9 | 10 | spill |
| c_1 | 10 | 17 | spill |
| d_1 | 11 | 18 | spill |
| r5 | 13 | 15 | x2 |
| r10 | 17 | 18 | x1 |
| r12 | 18 | 20 | x0 |

Current Start Interval = 18

**Active Set** = {(a,x0),(r10,x1)}

Remove a since its end is less than current start interval

**Active Set** = {((r10,x1),(r12,x0)}

Register allocation complete.